UNIVERSIDAD DE LA SALLE
ESCUELA DE CIENCIAS BÁSICAS
PROGRAMA DE CIENCIA DE DATOS

Manual de Programación y Preubas - Televentas

Presentado por:
Sebastián Gutiérrez Guayacundo

Docente:
RODRIGO ARANDA FERNANDEZ

Lenguaje de Programación II
Semestre III – 2026
Bogotá, Colombia

Contenido

1. Definición del problema real 3
2. Algoritmo en Python (Lógica de Solución) 3
3. Código en Visual Studio Code 6
4. Prueba de Escritorio (Caso: Compra de Smart TV) 8
Resultado: 8

1. Definición del problema real

La empresa "TeleVentas" requiere un sistema robusto para gestionar ventas a distancia. El problema técnico principal radica en la sincronización de procesos distribuidos: un cliente inicia una orden, pero el stock debe apartarse inmediatamente para evitar ventas de productos inexistentes. Además, el flujo debe ser estrictamente secuencial; no se puede enviar un producto (Logística) si el depósito no lo ha empacado, y el depósito no puede empacar si la pasarela de pagos (Tarjeta de Crédito) no ha confirmado la transacción. Finalmente, el sistema debe ser capaz de desviar incidencias (Quejas) directamente a la alta gerencia sin interrumpir el flujo de ventas.

2. Algoritmo en Python (Lógica de Solución)

El sistema se diseñó bajo un modelo de Flujo de Estados:

1. Carga de Inventario: Creación de objetos Producto.
2. Validación de Disponibilidad: Antes de agregar a la OrdenCompra, se verifica Producto.cantidad > 0.
3. Procesamiento de Pago: Invocación de la clase abstracta MetodoPago.
4. Actualización de Estatus: Cambio de estado de PENDIENTE a PAGADA.
5. Despacho y Envío: Delegación de la orden a las clases AgenteDeposito y Logistica.
6. Cierre de Gestión: Gestión de objetos Queja para soporte post-venta.

main.py

```
from modelos import Producto, OrdenCompra, TarjetaCredito, AgenteDeposito, Logistica, Queja

def simulacion_televentas():

# Usar un diseño de consola más limpio
    print("="*40)
    print("      SISTEMA TELEVENTAS LA SALLE")
    print("="*40)

    # 1. Catálogo e Inventario
    p1 = Producto("TV01", "Smart TV 50", 1800000, 10)

    # 2. El cliente crea orden y paga
    mi_orden = OrdenCompra("Sebastian Gutierrez")
    mi_orden.agregar_item(p1, 1)

    pago = TarjetaCredito()
    if pago.procesar_pago(1800000):
        mi_orden.estado = "PAGADA"

    # 3. Agente de Depósito (Consultando órdenes pagadas)
    agente = AgenteDeposito()
    agente.armar_pedido(mi_orden)
```

```
    # 4. Logística (Selección de transporte)
    Logistica.asignar_transporte(mi_orden, "Servientrega")

# 5. Soporte (Remisión inmediata a gerencia)
    print("\n" + "-"*40)
    reclamo = Queja("Sebastian G.", "Caja del TV un poco golpeada")
    reclamo.remitir()
    print("="*40)

if __name__ == "__main__":
    simulacion_televentas()
```

modelos.py

```
from abc import ABC, abstractmethod
from typing import List, Dict, Any

class Producto:
    def __init__(self, codigo: str, descripcion: str, precio: float, cantidad: int):
        self.codigo = codigo
        self.descripcion = descripcion
        self.precio = precio
        self.cantidad = cantidad

    def __str__(self) -> str:
        return f"[{self.codigo}] {self.descripcion} - ${self.precio} (Stock: {self.cantidad})"

class MetodoPago(ABC):
    @abstractmethod
    def procesar_pago(self, monto: float) -> bool:
        pass

class TarjetaCredito(MetodoPago):
    def procesar_pago(self, monto: float) -> bool:
        print(f"[Pago] Transacción exitosa por ${monto} con Tarjeta de Crédito.")
        return True

class OrdenCompra:
    def __init__(self, cliente: str):
        self.cliente = cliente
        self.items: List[Dict[str, Any]] = []
        self.estado = "PENDIENTE" # PENDIENTE, PAGADA, ARMADA, CANCELADA

    def agregar_item(self, producto: Producto, cantidad: int):
        if self.estado == "PENDIENTE" and producto.cantidad >= cantidad:
            self.items.append({"producto": producto, "cantidad": cantidad})
            producto.cantidad -= cantidad
            print(f"[Orden] Agregado: {producto.descripcion} x{cantidad}")
        else:
            print(f"[Error] No se puede agregar item. Estado: {self.estado} o sin stock.")
```

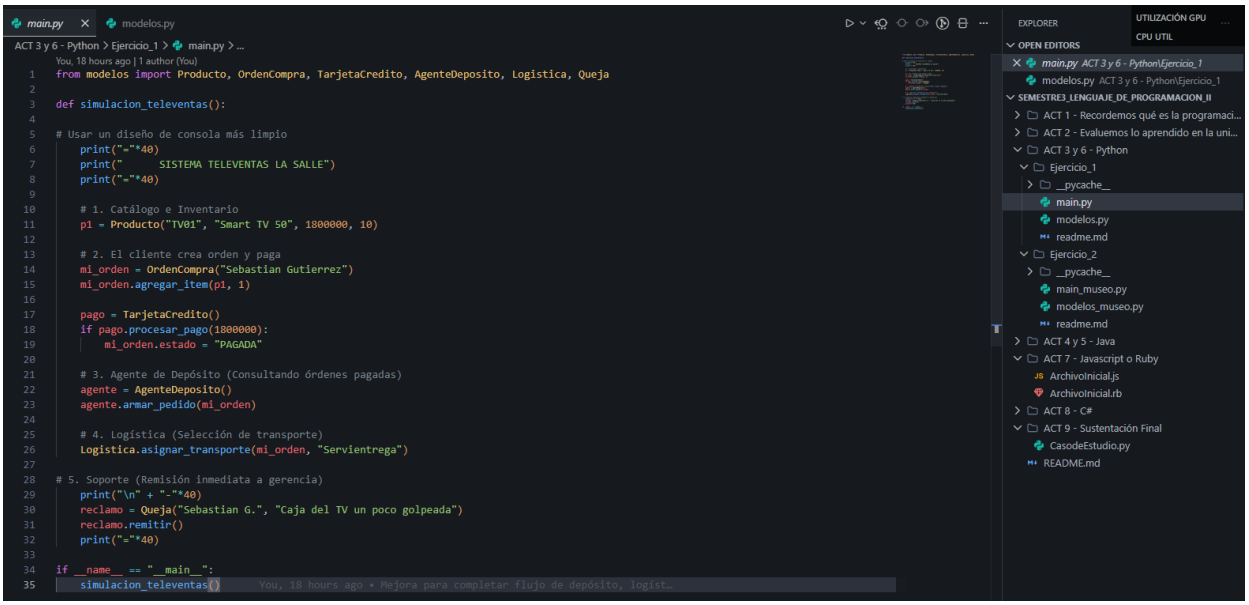
```
    def cancelar_orden(self):
        if self.estado == "PENDIENTE" or self.estado == "PAGADA":
            # Devolvemos el stock al sistema de inventario
            for item in self.items:
                item["producto"].cantidad += item["cantidad"]
            self.estado = "CANCELADA"
            print(f"[Orden] La orden de {self.cliente} ha sido cancelada.")

class AgenteDeposito:
    def armar_pedido(self, orden: OrdenCompra):
        if orden.estado == "PAGADA":
            print(f"[Depósito] Armando y empaquetando pedido de: {orden.cliente}")
            orden.estado = "ARMADA"
        else:
            print("[Depósito] La orden aún no ha sido pagada.")

class Logistica:
    @staticmethod
    def asignar_transporte(orden: OrdenCompra, empresa: str):
        if orden.estado == "ARMADA":
            print(f"[Logística] Pedido delegado a la empresa: {empresa}")
            orden.estado = "ENVIADA"
        else:
            print("[Logística] El pedido debe estar armado antes de asignar transporte.")

class Queja:
    def __init__(self, cliente: str, motivo: str):
        self.cliente = cliente
        self.motivo = motivo

    def remitir(self):
        print(f"[Gerencia] RECLAMO RECIBIDO - Cliente: {self.cliente} | Motivo: {self.motivo}")
```

3. Código en Visual Studio Code

main.py

Este archivo funciona como el controlador. Su función es:

1. **Instanciar objetos:** Crea los productos y usuarios iniciales.
2. **Ejecutar el Flujo:** Coordina la secuencia lógica (Venta -> Pago -> Empaque -> Envío).
3. **Manejo de Estados:** Es el encargado de verificar si el pago fue exitoso para actualizar la variable mi_orden.estado, disparando así el resto de los procesos del depósito.



modelos.py

Este archivo funciona como el "motor" del sistema. Se construyó utilizando los siguientes pilares de la POO:

- **Clases de Entidad (Producto, Queja):** Almacenan los datos básicos y su representación textual.

- **Abstracción (MetodoPago):** Se definió una clase abstracta para que el sistema de cobro sea genérico. Esto permite que el software sea escalable; hoy usamos tarjeta de crédito, pero se pueden añadir otros medios sin modificar el código de la orden.

- **Lógica de Negocio (OrdenCompra):** Es la clase principal. Controla el flujo de estados y garantiza la integridad del inventario. Por ejemplo, el método agregar_item contiene la lógica para descontar stock automáticamente, y cancelar_orden contiene el algoritmo para devolverlo al sistema.

- **Servicios Especializados (AgenteDeposito, Logistica):** Clases que actúan como departamentos del mundo real. Logistica usa un método estático (@staticmethod) porque su función es de servicio delegado y no requiere guardar datos internos del transporte.

main.py | modelos.py

ACT 3 y 6 - Python > Ejercicio_1 > modelos.py > Producto

You, 18 hours ago | 1 author (You)

```python
from abc import ABC, abstractmethod
from typing import List, Dict, Any

You, 18 hours ago | 1 author (You)
class Producto:
    def __init__(self, codigo: str, descripcion: str, precio: float, cantidad: int):
        self.codigo = codigo
        self.descripcion = descripcion
        self.precio = precio
        self.cantidad = cantidad
        You, 3 days ago • Ajuste en modelos.py debido a error de ejecución…
    def __str__(self) -> str:
        return f"[{self.codigo}] {self.descripcion} - ${self.precio} (Stock: {self.cantidad})"

You, 18 hours ago | 1 author (You)
class MetodoPago(ABC):
    @abstractmethod
    def procesar_pago(self, monto: float) -> bool:
        pass

You, 18 hours ago | 1 author (You)
class TarjetaCredito(MetodoPago):
    def procesar_pago(self, monto: float) -> bool:
        print(f"[Pago] Transacción exitosa por ${monto} con Tarjeta de Crédito.")
        return True

You, 18 hours ago | 1 author (You)
class OrdenCompra:
    def __init__(self, cliente: str):
        self.cliente = cliente
        self.items: List[Dict[str, Any]] = []
        self.estado = "PENDIENTE" # PENDIENTE, PAGADA, ARMADA, CANCELADA

    def agregar_item(self, producto: Producto, cantidad: int):
        if self.estado == "PENDIENTE" and producto.cantidad >= cantidad:
            self.items.append({"producto": producto, "cantidad": cantidad})
            producto.cantidad -= cantidad
            print(f"[Orden] Agregado: {producto.descripcion} x{cantidad}")
        else:
            print(f"[Error] No se puede agregar item. Estado: {self.estado} o sin stock.")

    def cancelar_orden(self):
        if self.estado == "PENDIENTE" or self.estado == "PAGADA":
            # Devolvemos el stock al sistema de inventario
            for item in self.items:
                item["producto"].cantidad += item["cantidad"]
            self.estado = "CANCELADA"
            print(f"[Orden] La orden de {self.cliente} ha sido cancelada.")
```

EXPLORER | UTILIZACIÓN GPU | CPU UTIL 13 %

OPEN EDITORS
- main.py ACT 3 y 6 - Python\Ejercicio_1
- modelos.py ACT 3 y 6 - Python\Ejercicio_1

SEMESTRE3_LENGUAJE_DE_PROGRAMACION_II
- ACT 1 - Recordemos qué es la programaci...
- ACT 2 - Evaluemos lo aprendido en la uni...
- ACT 3 y 6 - Python
  - Ejercicio_1
    - __pycache__
    - main.py
    - modelos.py
    - readme.md
  - Ejercicio_2
    - __pycache__
    - main_museo.py
    - modelos_museo.py
    - readme.md
- ACT 4 y 5 - Java
- ACT 7 - Javascript o Ruby
  - ArchivoInicial.js
  - ArchivoInicial.rb
- ACT 8 - C#
- ACT 9 - Sustentación Final
  - CasodeEstudio.py
  - README.md

main.py | modelos.py

ACT 3 y 6 - Python > Ejercicio_1 > modelos.py > Producto

```python
class OrdenCompra:
    def agregar_item(self, producto: Producto, cantidad: int):
        self.items.append({"producto": producto, "cantidad": cantidad})
        producto.cantidad -= cantidad
        print(f"[Orden] Agregado: {producto.descripcion} x{cantidad}")
    else:
        print(f"[Error] No se puede agregar item. Estado: {self.estado} o sin stock.")

    def cancelar_orden(self):
        if self.estado == "PENDIENTE" or self.estado == "PAGADA":
            # Devolvemos el stock al sistema de inventario
            for item in self.items:
                item["producto"].cantidad += item["cantidad"]
            self.estado = "CANCELADA"
            print(f"[Orden] La orden de {self.cliente} ha sido cancelada.")

You, 18 hours ago | 1 author (You)
class AgenteDeposito:
    def armar_pedido(self, orden: OrdenCompra):
        if orden.estado == "PAGADA":
            print(f"[Depósito] Armando y empaquetando pedido de: {orden.cliente}")
            orden.estado = "ARMADA"
        else:
            print("[Depósito] La orden aún no ha sido pagada.")

You, 18 hours ago | 1 author (You)
class Logistica:
    @staticmethod
    def asignar_transporte(orden: OrdenCompra, empresa: str):
        if orden.estado == "ARMADA":
            print(f"[Logística] Pedido delegado a la empresa: {empresa}")
            orden.estado = "ENVIADA"
        else:
            print("[Logística] El pedido debe estar armado antes de asignar transporte.")

You, 18 hours ago | 1 author (You)
class Queja:
    def __init__(self, cliente: str, motivo: str):
        self.cliente = cliente
        self.motivo = motivo

    def remitir(self):
        print(f"[Gerencia] RECLAMO RECIBIDO - Cliente: {self.cliente} | Motivo: {self.motivo}")
```

EXPLORER | UTILIZACIÓN GPU | CPU UTIL

OPEN EDITORS
- main.py ACT 3 y 6 - Python\Ejercicio_1
- modelos.py ACT 3 y 6 - Python\Ejercicio_1

SEMESTRE3_LENGUAJE_DE_PROGRAMACION_II
- ACT 1 - Recordemos qué es la programaci...
- ACT 2 - Evaluemos lo aprendido en la uni...
- ACT 3 y 6 - Python
  - Ejercicio_1
    - __pycache__
    - main.py
    - modelos.py
    - readme.md
  - Ejercicio_2
    - __pycache__
    - main_museo.py
    - modelos_museo.py
    - readme.md
- ACT 4 y 5 - Java
- ACT 7 - Javascript o Ruby
  - ArchivoInicial.js
  - ArchivoInicial.rb
- ACT 8 - C#
- ACT 9 - Sustentación Final
  - CasodeEstudio.py
  - README.md

4. Prueba de Escritorio (Caso: Compra de Smart TV)

Línea de Código	Acción Realizada	producto.cantidad	orden.estado
p1 = Producto(...)	Inicialización	10	N/A
mi_orden.agregar_item(p1, 1)	Se resta el stock	9	PENDIENTE
if pago.procesar_pago(...)	Validación de pago	9	PAGADA
agente.armar_pedido(...)	Preparación física	9	ARMADA
Logistica.asignar(...)	Despacho final	9	ENVIADA

Resultado:

```
sebas@SoyySebas MINGW64 ~/Semestre3_Lenguaje_de_Programacion_II (main)
$ C:/Users/sebas/AppData/Local/Programs/Python/Python312/python.exe "c:/Users/sebas/Semestre3_Lenguaje_de_Programacion_II/ACT 3 y 6 - Python/Ejercicio_1/main.py"
========================================
      SISTEMA TELEVENTAS LA SALLE
========================================
[Orden] Agregado: Smart TV 50 x1
[Pago] Transacción exitosa por $1800000 con Tarjeta de Crédito.
[Depósito] Armando y empaquetando pedido de: Sebastian Gutierrez
[Logística] Pedido delegado a la empresa: Servientrega

----------------------------------------
[Gerencia] RECLAMO RECIBIDO - Cliente: Sebastian G. | Motivo: Caja del TV un poco golpeada
========================================
```

Tras la ejecución de la simulación y el análisis de la trazabilidad en consola, se pueden extraer las siguientes conclusiones:

- **Integridad del Proceso de Negocio:** El sistema garantiza un flujo secuencial lógico. Como se observa en el log, la Logística solo se activa después de que el Depósito confirma el empaque, y este a su vez solo actúa tras una Transacción exitosa. Esto demuestra un control efectivo de los estados del objeto OrdenCompra.

- **Gestión Eficiente de Inventario:** El mensaje [Orden] Agregado confirma que la comunicación entre la clase OrdenCompra y Producto es bidireccional, permitiendo la reserva inmediata del stock para evitar ventas en exceso (overselling).

- **Desacoplamiento de Soporte (Escalabilidad):** El sistema de Quejas funciona de manera independiente al flujo de despacho. El resultado muestra que, aunque el pedido ya fue entregado a la transportadora, el módulo de soporte puede capturar incidencias y remitirlas a la Gerencia sin afectar la base de datos de ventas o logística.

- **Eficacia de la Abstracción:** El uso de una clase abstracta para el pago permitió que el sistema procesara la Tarjeta de Crédito de forma transparente, validando que la arquitectura es flexible para integrar nuevos métodos de pago en futuras fases del proyecto.